

Boundary Design Patterns

Series: IAM | **Notebook:** 5 of 10 | **Created:** January 2026 | **Last Updated:** 04/04/2026

Controlling Data Visibility with Boundaries

Boundaries determine **what data** users can see. While policies control actions, boundaries filter visibility. This notebook covers boundary syntax, patterns, and implementation strategies.

Table of Contents

- 1. [Boundary Fundamentals](#)
- 2. [The Three-Domain Model](#)
- 3. [Boundary Syntax Reference](#)
- 4. [Security Context and Data Partitioning Strategy](#)
- 5. [Common Boundary Patterns](#)
- 6. [Multi-Tenant Isolation](#)
- 7. [Boundary Testing](#)

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS with Gen3 IAM enabled
Permissions	<code>environment-admin</code> or boundary management rights

Requirement	Details
Prior Knowledge	IAM-01 through IAM-04

1. Boundary Fundamentals

Boundaries filter what entities and data a user can see within an environment.

Policies vs Boundaries

Concept	Controls	Example
Policy	What actions	"Can read logs"
Boundary	What data	"Logs from checkout service only"

A user needs BOTH:

- A **policy** granting the action (e.g., `storage:logs:read`)
- A **boundary** including the data (e.g., `checkout` security context)

How Boundaries Work

1. User attempts to access data (query, view, etc.)
2. Dynatrace checks if user's policy allows the action
3. Dynatrace filters results to match user's boundary
4. User sees only data within their boundary

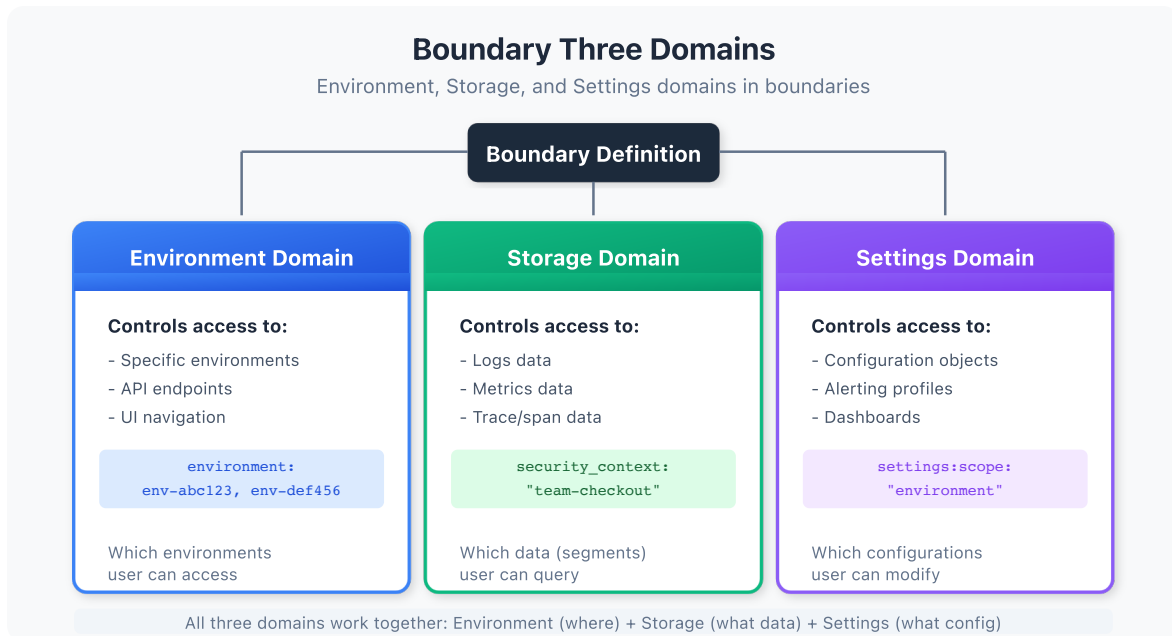
Boundary Scope

Boundaries are **environment-level** and assigned to groups:

```
Group: dt-checkout-editors
├─ Policy: environment-editor
└─ Boundary: checkout-services-only
```

2. The Three-Domain Model

Boundaries filter across three domains. A complete boundary typically includes all three.



Domain 1: Environment

Controls visibility of Smartscape entities:

- Hosts
- Services
- Applications
- Databases
- Cloud resources

Domain 2: Storage

Controls visibility of Grail data:

- Logs
- Spans (traces)
- Metrics
- Events
- Business events

Domain 3: Settings

Controls visibility of configuration:

- Alerting rules
- SLOs
- Custom settings

Why All Three Matter

If you only boundary one domain:

Missing Domain	Problem
Environment	User can't see services in UI
Storage	User can't query logs/spans
Settings	User can't see/edit config

3. Boundary Syntax Reference

Basic Boundary Structure

```
<domain>:<field> <operator> (<values>)
```

Complete Three-Domain Boundary:

```
environment:management-zone IN ("checkout");  
storage:dt.security_context IN ("checkout");  
settings:dt.security_context IN ("checkout");
```

Operators

Operator	Description	Example
IN	Match any in list	IN ("a", "b", "c")
=	Exact match	= "checkout"
!=	Not equal	!= "restricted"

Operator	Description	Example
<code>startsWith</code>	Prefix match	<code>startsWith "team-"</code>
<code>contains</code>	Substring	<code>contains "prod"</code>
<code>MATCH</code>	Wildcard pattern match	<code>storage:dt.security_context</code> <code>MATCH('*/app:easytrade')</code>

`MATCH()` is available for storage domain only. It supports `*` as a wildcard at any position. Classic entity access (`environment: domain`) does not support `MATCH()` — use `startsWith` or `IN` there instead.

Common Fields

Domain	Field	Description
environment	<code>management-zone</code>	Management zone filter
storage	<code>dt.security_context</code>	Data security context
storage	<code>bucket-name</code>	Storage bucket name
settings	<code>dt.security_context</code>	Settings security context
settings	<code>schemaId</code>	Settings schema

Multiple Values

Grant access to multiple security contexts:

```
environment:management-zone IN ("checkout", "payments", "shared");
storage:dt.security_context IN ("checkout", "payments", "shared");
settings:dt.security_context IN ("checkout", "payments", "shared");
```

Wildcard Access

For admin groups needing full access:

```
environment:management-zone IN ("*");
storage:dt.security_context IN ("*");
settings:dt.security_context IN ("*");
```

Boundary Limitations

Limitation	Description	Workaround
Max 10 conditions	Only 10 lines per boundary	Create multiple boundaries
No AND between lines	Each line is OR-combined	Use multiple boundaries for AND logic
MATCH storage-only	<code>MATCH()</code> not supported in <code>environment: domain</code>	Use <code>startsWith</code> / <code>IN</code> for Classic entities

4. Security Context and Data Partitioning Strategy

Security contexts and Grail buckets form the foundation of boundary filtering and data partitioning. Plan your strategy carefully.

What is a Security Context?

A **security context** is a label applied to:

- Entities (hosts, services)
- Data (logs, spans)
- Settings (configurations)

Boundaries filter based on these labels.

Primary Grail Fields

Primary Grail fields are first-class attributes that Dynatrace uses for data organization. These should drive your bucket, permission, and segmentation strategy:

Field	Source	Use Case
<code>k8s.cluster.name</code>	Kubernetes	Cluster-level isolation

Field	Source	Use Case
<code>k8s.namespace.name</code>	Kubernetes	Namespace-level access
<code>aws.account.id</code>	AWS metadata	AWS account separation
<code>azure.subscription.id</code>	Azure metadata	Azure subscription separation
<code>gcp.project.id</code>	GCP metadata	GCP project separation
<code>dt.host_group.id</code>	OneAgent config	Host group-based isolation

These fields are automatically enriched from infrastructure metadata and provide consistent, reliable partitioning dimensions.

Grail Bucket Strategy

Buckets partition data in Grail for access control, retention, and query performance.

Key Principle: Set up buckets along organizational lines and route data based on primary Grail fields.

Bucket Design	Example	Benefit
By Environment	<code>prod_logs</code> , <code>dev_logs</code>	Separate prod/non-prod access
By Team/LOB	<code>checkout_data</code> , <code>payments_data</code>	Team-level isolation
By Compliance	<code>pci_logs</code> , <code>general_logs</code>	Regulatory separation
By Region	<code>us_data</code> , <code>eu_data</code>	Geographic compliance

Bucket Limitations

 **Critical:** Plan buckets carefully - these constraints cannot be changed later.

Limitation	Details
One data type per bucket	Logs, metrics, events, OR spans - not mixed
Names are immutable	Bucket names CANNOT be changed after creation

Limitation	Details
No data migration	Data CANNOT be moved between buckets
Naming rules	3-100 chars, lowercase alphanumeric, underscores, hyphens only
Maximum buckets	80 per environment (default limit)
Optimal ingest	~1 TB/day per bucket for best query performance
Acceptable ingest	1-3 TB/day per bucket (limited query window)
Maximum ingest	3 TB/day per bucket hard limit

Query Constraints

Limit	Value	Impact
Maximum data scanned	500 GB	Limits queryable time window
Maximum records returned	1,000	Use aggregations for larger datasets
Maximum response payload	1 MB	Large result sets may be truncated

Default Bucket Retentions

Bucket	Retention
<code>default_logs</code>	35 days
<code>default_metrics</code>	15 months
<code>default_spans</code>	10 days
<code>default_events</code>	35 days

Naming Convention: Include organization, data type, and retention in bucket names:

- `teamA_logs_90d`
- `prod_spans_30d`
- `pci_metrics_365d`

Bucket + Boundary Integration

Use `storage:bucket-name` in boundaries to restrict team access to their data:

```
// Policy: Grant read access to team's bucket
ALLOW storage:logs:read WHERE storage:bucket-name = "checkout_logs"
ALLOW storage:spans:read WHERE storage:bucket-name = "checkout_spans"

// Boundary: Restrict to team's buckets and security context
storage:bucket-name IN ("checkout_logs", "checkout_spans");
storage:dt.security_context IN ("checkout");
environment:management-zone IN ("Checkout");
```

Using Buckets for Team Isolation

For team-level data access control:

1. **Create team-specific buckets** - `teamA_logs`, `teamA_spans`, etc.
2. **Configure pipeline routing** - Route data to buckets based on primary Grail fields
3. **Create bucket-based policies** - Grant access to specific buckets
4. **Apply boundaries** - Combine bucket + security context restrictions

```
// Complete team isolation example
Group: Checkout-Team
├─ Policy: Checkout Data Access
│   ├── ALLOW storage:logs:read WHERE storage:bucket-name = "checkout_logs"
│   ├── ALLOW storage:spans:read WHERE storage:bucket-name = "checkout_spans"
│   └─ ALLOW storage:metrics:read WHERE storage:bucket-name =
"checkout_metrics"
└─ Boundary:
    storage:bucket-name IN ("checkout_logs", "checkout_spans",
"checkout_metrics");
    storage:dt.security_context IN ("checkout");
    environment:management-zone IN ("Checkout");
```

For comprehensive bucket guidance, see **ORGNZ-03: Bucket Strategy and Design** which covers naming conventions, retention planning, and cost attribution patterns.

Primary Grail Tags

Beyond fields, **Primary Grail Tags** can be configured from:

- Kubernetes labels (first 3 selected during setup)
- AWS resource tags
- Azure resource tags

These tags become first-class Grail attributes available for:

- Bucket assignment rules
- Policy/boundary conditions
- Pipeline routing
- Segment filters

Assigning Security Context

Security context is set via (modern approaches):

1. **Entity Enrichment rules** - Automatic based on entity properties
2. **Host properties** - Set on hosts and inherited by services
3. **Primary Grail fields/tags** - First-class tags for Grail data
4. **OneAgent group** - Inherited from deployment
5. **Log attributes** - Set during OpenPipeline ingestion

Note: Auto-tagging is a legacy approach. Prefer Entity Enrichment and Primary Grail Fields for new implementations.

Security Context Design Patterns

Pattern	Example Values	Use Case
Team-based	checkout-team , payments-team	Team ownership
Application-based	ecommerce-app , mobile-app	App isolation
Environment-based	prod , staging , dev	Env separation
Business unit	retail , wholesale , corporate	Business isolation
Geography	us-east , eu-west , apac	Regional separation

Naming Conventions

Rule	Good	Bad
Lowercase	checkout	Checkout
Hyphens	team-checkout	team_checkout
Descriptive	payments-api	pa
No spaces	mobile-app	mobile app

Shared Context

Some data should be visible to multiple teams:

Security Contexts:

- └ checkout (checkout team data)
- └ payments (payments team data)
- └ shared (cross-team data - infrastructure, common services)

Teams get their context + `shared` :

```
environment:management-zone IN ("checkout", "shared");
```

Recommended Partitioning Strategy

1. **Identify primary dimensions** - What drives access control? (team, environment, region, compliance)
2. **Map to Primary Grail Fields** - Use `k8s.cluster.name` , `aws.account.id` , `dt.host_group.id`
3. **Design buckets** - Create buckets aligned with access control boundaries
4. **Configure pipelines** - Route data to buckets based on primary fields
5. **Define policies** - Grant bucket-specific permissions per team
6. **Apply boundaries** - Use `storage:bucket-name` and `storage:dt.security_context`
7. **Create segments** - For cross-bucket query filtering

See Also: For migration from Management Zones, refer to **MZ2POL-04: Policies**

and **Boundaries** which covers mapping MZ patterns to the new model.

5. Common Boundary Patterns

Pattern 1: Single Team Boundary

Restrict to one team's data:

```
environment:management-zone IN ("checkout");  
storage:dt.security_context IN ("checkout");  
settings:dt.security_context IN ("checkout");
```

Pattern 2: Team + Shared

Team data plus shared infrastructure:

```
environment:management-zone IN ("checkout", "shared", "infrastructure");  
storage:dt.security_context IN ("checkout", "shared", "infrastructure");  
settings:dt.security_context IN ("checkout", "shared");
```

Pattern 3: Multiple Teams (Cross-Functional)

For SRE or platform teams:

```
environment:management-zone IN ("checkout", "payments", "catalog", "shared");  
storage:dt.security_context IN ("checkout", "payments", "catalog", "shared");  
settings:dt.security_context IN ("checkout", "payments", "catalog",  
"shared");
```

Pattern 4: Environment Tier

Production vs non-production:

Production:

```
environment:management-zone IN ("prod-checkout", "prod-payments");  
storage:dt.security_context IN ("prod-checkout", "prod-payments");  
settings:dt.security_context IN ("prod-checkout", "prod-payments");
```

Non-Production:

```
environment:management-zone IN ("dev-checkout", "staging-checkout");
storage:dt.security_context IN ("dev-checkout", "staging-checkout");
settings:dt.security_context IN ("dev-checkout", "staging-checkout");
```

Pattern 5: Read-All, Write-Scoped

Using two groups for the same user:

Group 1: All-Viewers (broad read)

```
Policy: environment-viewer
Boundary: environment:management-zone IN ("*");
```

Group 2: Checkout-Editors (scoped write)

```
Policy: checkout-write-policy
Boundary: environment:management-zone IN ("checkout");
```

Pattern 6: Application Team — Structured Security Context

When `dt.security_context` follows the `comp:<component>/bu:<bu>/app:<app>` format (see **Structured Security Context Design** in **IAM-04**), app teams can access all component types for their application using a single wildcard boundary:

```
// 3rd Gen signals and entities – wildcard on app dimension
storage:dt.security_context MATCH('*/app:easytrade');

// Classic entities – explicit per-component prefix (startsWith not supported
in environment: domain)
storage:dt.security_context startsWith('comp:app/bu:digital/app:easytrade');
storage:dt.security_context startsWith('comp:db/bu:digital/app:easytrade');
storage:dt.security_context startsWith('comp:lb/bu:digital/app:easytrade');
```

This grants access to all data tagged with any component prefix for that application:

- `comp:app/bu:digital/app:easytrade`
- `comp:db/bu:digital/app:easytrade`
- `comp:lb/bu:digital/app:easytrade`

Pattern 7: Transversal Team — Component-Based Access

Infrastructure teams (database, network, OS) that need cross-application access to their component type use a leading `comp:` prefix to enable a single rule:

```
// 3rd Gen signals and entities – all database components, all applications
storage:dt.security_context MATCH('comp:db*');

// Classic entities
storage:dt.security_context startsWith('comp:db');
```

This grants access to all data matching:

- `comp:db/bu:digital/app:easytrade`
- `comp:db/bu:digital/app:easytravel`
- `comp:db/bu:corp/app:hipstershop`

Dimension order is the key design decision. Placing `comp` first in the string means component-based transversal access is always a simple prefix match. Access by `bu` or other mid-string dimensions requires `MATCH('*/bu:digital/*')` for Grail and explicit per-component rules for Classic entities.

Multi-Value Security Context (Upcoming)

Planned: CQ3 — `PRODUCT-14849`

The current single-string model requires choosing one leading dimension. A future **multi-value** `dt.security_context` will store each dimension independently:

```
dt.security_context = ["bu:digital", "app:easytrade", "comp:db"]
```

This removes the dimension-ordering trade-off and simplifies all three team types to a single boundary condition per signal type:

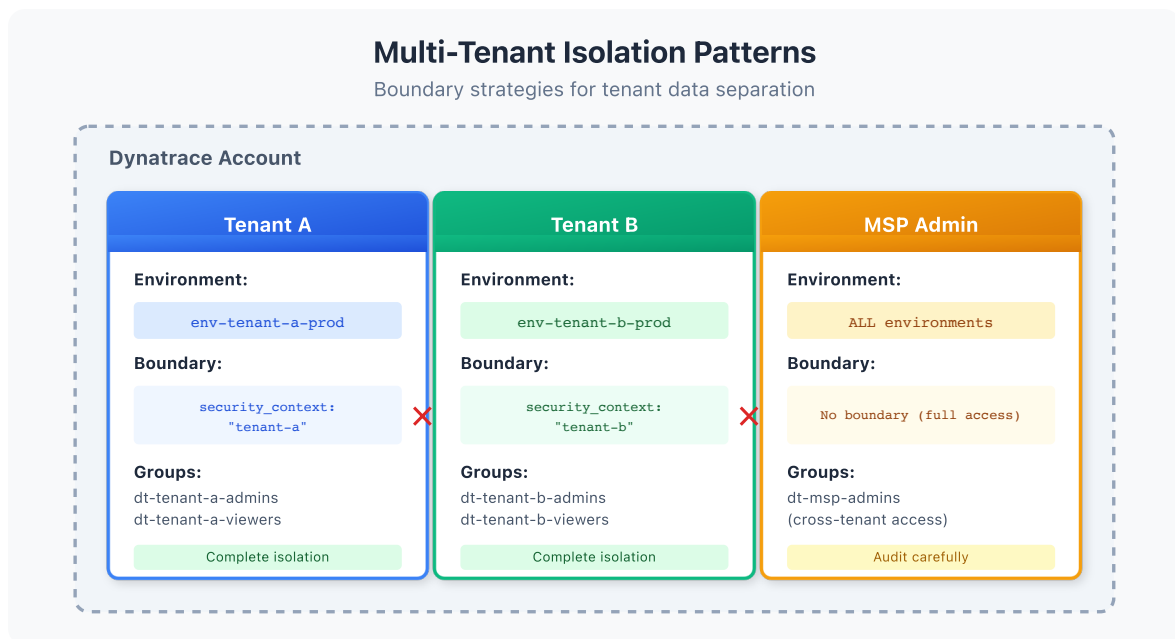
Team	Boundary (future)
App team (easytrade)	<code>storage:dt.security_context</code> <code>MATCH('app:easytrade')</code>

Team	Boundary (future)
Database team (transversal)	<code>storage:dt.security_context MATCH('comp:db')</code>
Business unit (digital)	<code>storage:dt.security_context MATCH('bu:digital')</code>

Until multi-value is available, the structured single-string pattern above is the recommended approach.

6. Multi-Tenant Isolation

For organizations serving multiple customers or business units that require strict isolation.



Isolation Requirements

Requirement	Implementation
Data isolation	Unique security context per tenant
No cross-tenant access	Strict boundary matching
Audit capability	Centralized logging
Shared infrastructure	Separate "platform" context

Multi-Tenant Boundary Example

Tenant A Group:

```
environment:management-zone IN ("tenant-a");
storage:dt.security_context IN ("tenant-a");
settings:dt.security_context IN ("tenant-a");
```

Tenant B Group:

```
environment:management-zone IN ("tenant-b");
storage:dt.security_context IN ("tenant-b");
settings:dt.security_context IN ("tenant-b");
```

Platform Team (cross-tenant):

```
environment:management-zone IN ("platform", "shared");
storage:dt.security_context IN ("platform", "shared");
settings:dt.security_context IN ("platform", "shared");
```

Entity Enrichment for Multi-Tenancy

Use Entity Enrichment rules (Settings > Entity Enrichment) to assign tenant context:

```
# Entity Enrichment rule based on host group
Rule: Assign security context from host group
Entity Type: Host
Condition: Host group name contains "tenant-"
Action: Set dt.security_context = {hostGroup.name}
```

Alternatively, use **host properties** to set context at deployment:

```
# Set via OneAgent installer
--set-host-property=dt.security_context=tenant-a
```

7. Boundary Testing

Verify boundaries work as expected before production use.


```
// Check security context distribution on services
fetch dt.entity.service
| summarize count = count(), by:{dt.security_context}
| sort count desc
| limit 20

// Alternative: Smartscape on Grail (entity.name → name)
// smartscapeNodes SERVICE
// | summarize count = count(), by:{dt.security_context}
// | sort count desc
// | limit 20
```

```
// Find entities without security context (boundary gaps)
fetch dt.entity.service
| filter isNull(dt.security_context)
| fields entity.name, tags
| sort entity.name
| limit 50
```

```
// Check host security context coverage
fetch dt.entity.host
| summarize
    total = count(),
    withContext = countIf(isNotNull(dt.security_context)),
    missing = countIf(isNull(dt.security_context))
| fieldsAdd coveragePercent = round(100.0 * withContext / total, decimals: 2)

// Alternative: Smartscape on Grail (entity.name → name)
// smartscapeNodes HOST
// | summarize
// total = count(),
// withContext = countIf(isNotNull(dt.security_context)),
// missing = countIf(isNull(dt.security_context))
// | fieldsAdd coveragePercent = round(100.0 * withContext / total, decimals: 2)
```

```
// Verify logs have security context
fetch logs, from: now() - 1h
| summarize
    total = count(),
    withContext = countIf(isNotNull(dt.security_context))
| fieldsAdd coveragePercent = round(100.0 * withContext / total, decimals: 2)
```

Testing Methodology

1. **Create test group** with the boundary
2. **Add test user** to the group
3. **Log in as test user** (or impersonate)
4. **Verify visible entities** match expected
5. **Verify hidden entities** are not visible
6. **Query data** to confirm filtering works

Validation Checklist

Test	Expected
List services	Only boundary-included services
Query logs	Only logs with matching context
View settings	Only settings with matching context
Access denied entity	Empty result or error

Next Steps

With boundaries configured, complete your IAM implementation:

Recommended Path

1. **IAM-06: User Lifecycle and Provisioning** - Automate user management
2. **IAM-07: Audit Logging and Compliance** - Monitor access patterns
3. **IAM-08: Multi-Environment IAM** - Scale across environments

Boundary Checklist

Before moving on, ensure you have:

- ☐ Understood the three-domain model
- ☐ Designed your security context strategy
- ☐ Created boundaries for each team/group

- [] Configured entity enrichment for context assignment
 - [] Tested boundaries with real users
 - [] Verified no entities are missing context
-

Summary

In this notebook, you learned:

- Boundary fundamentals and how they differ from policies
 - The three-domain model (environment, storage, settings)
 - Boundary syntax and operators
 - Security context strategy and naming
 - Five common boundary patterns
 - Multi-tenant isolation design
 - Boundary testing and validation
-

References

- [Permission Boundaries](#)
 - [Security Context](#)
 - [Entity Enrichment](#)
 - [Host Properties](#)
-

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.